



LECTURE 15

GRAPH EMBEDDINGS: GETTING STRUCTURED KNOWLEDGE INTO NEURAL SPACE

DATASCI290: NEUROSymbolic AI
MARCH 2 2026

REFERENCES

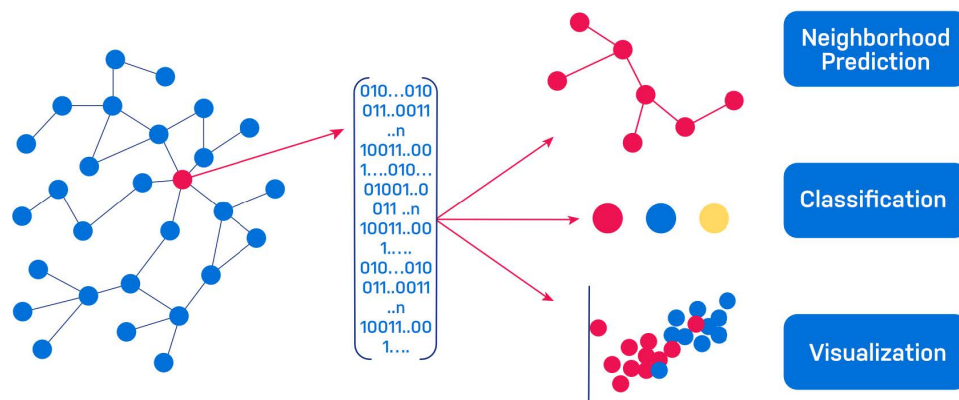
■ Node2Vec

- Grover, A., & Leskovec, J. (2016, August). node2vec: Scalable feature learning for networks. In Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining (pp. 855-864).

■ TransE

- Bordes, A., Usunier, N., Garcia-Duran, A., Weston, J., & Yakhnenko, O. (2013). Translating embeddings for modeling multi-relational data. *Advances in neural information processing systems*, 26.

WHY: PRESERVE NETWORK/GRAPH/KNOWLEDGE GRAPH NEIGHBORHOODS



- **Graph embeddings = learned coordinates for graph objects**
 - Map each node (and sometimes edges/relations) to a vector $z_v \in R^d$
 - Learned so that **structure is preserved** (neighbors, communities, roles, or relation patterns)
- **Core idea:** make “who is related to whom” become **distances + directions in vector space**
 - Similar nodes → close vectors
 - Predictable relationships → consistent geometric patterns
- **graphs → vectors → machine learning**
- One representation supports many tasks: classification, retrieval, clustering, ranking, anomaly hints
- Often beats hand-crafted graph features when graphs are large/complex

EMBEDDINGS

- (Word/Text) embeddings have been about **similarity**
-
- *So what notion of “similarity” do we want the (graph) embedding to capture ?*

SIMILARITIES

- **Direct connection similarity (local proximity)** Similar if they are linked or share many immediate neighbors
- **Community / group similarity (same cluster)** Similar if they live in the same densely connected region, even if not directly linked
- **Role / structural similarity (same “job” in the network)** Similar if they play the same structural role (hub, bridge, peripheral), even if far apart
- **Functional similarity (similar behavior over relations)** Similar if they connect to similar *types* of things in similar ways
- **Path-based / reachability similarity (how they connect through multi-hop paths)** Similar if they have similar multi-step relationships or can reach the same kinds of nodes
- **Semantic similarity in typed knowledge graphs (meaning of relations matters)** Similar if they participate in similar relation patterns (not just connectivity)
- Depends on the **TASK**

SIMILARITIES

- **Direct connection similarity (local proximity)** Similar if they are linked or share many immediate neighbors
 - *Examples:* friends in a social graph; proteins that interact; devices that communicate frequently
- **Community / group similarity (same cluster)** Similar if they live in the same densely connected region, even if not directly linked
 - *Examples:* social circles; research sub-communities in a citation network; customer segments in a product graph
- **Role / structural similarity (same “job” in the network)** Similar if they play the same structural role (hub, bridge, peripheral), even if far apart
 - *Examples:* “influencers” across different social circles; airports that are major hubs; routers that are gateways between subnets
- **Functional similarity (similar behavior over relations)** Similar if they connect to similar *types* of things in similar ways
 - *Examples:* drugs that target similar pathways; companies with similar supplier/customer patterns; users with similar sequences of actions
- **Path-based / reachability similarity (how they connect through multi-hop paths)** Similar if they have similar multi-step relationships or can reach the same kinds of nodes
 - *Examples:* “people you may know” via mutuals; fraud rings connected through intermediaries; gene → disease associations via pathways
- **Semantic similarity in typed knowledge graphs (meaning of relations matters)** Similar if they participate in similar relation patterns (not just connectivity)
 - *Examples:* in a medical KG, two diseases similar because they share symptoms and treatments; in an enterprise KG, two vendors similar because they supply similar parts and have similar compliance attributes

WHY NOT DEAL WITH KNOWLEDGE PURELY SYMBOLICALLY ?

- Real-world Knowledge Graphs are (typically)
 - **Big**
 - **Noisy**
 - **Incomplete**



NODE2VEC



TURNING A GRAPH INTO SEQUENCES: TAKE A (RANDOM) WALK

- **Text = Sequence** of words (tokens)
- **Core problem:** graphs have no natural order → we must *construct* node “contexts”
- A “**random walk**” :
 - Start at node u
 - Repeatedly move to one of its neighbors (randomly)
 - The visited nodes form a sequence
 - These sequences act like “sentences” of nodes
 - Repeating many walks builds a corpus of structural contexts
- Learned Embeddings: nodes appearing in **similar walk contexts get similar vectors**

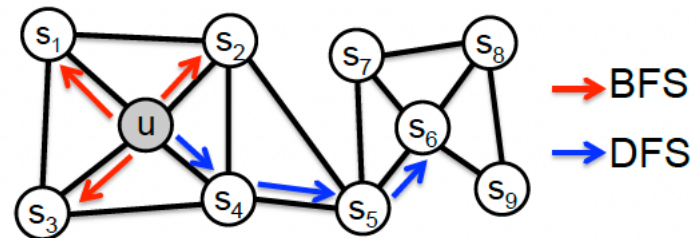
SEARCH STRATEGIES

■ Breadth-First Search (BFS)

- Explores neighbors level by level
- Stays close to the starting node
- Captures *local neighborhood structure*

■ Depth-First Search (DFS)

- Explores as far as possible before backtracking
 - Moves outward into distant parts of the graph
 - Captures *longer-range structure*
- With the same sampling budget, BFS spends it “near,” DFS spends it “far”
 - The way we sample neighborhoods determines what similarity the embedding will learn



2ND-ORDER RANDOM WALKS: CONTROLLED EXPLORATION

- A standard random walk depends only on the current node
- A **2nd-order random walk** remembers the previous step
 - Next move depends on (previous node $\rightarrow$ current node $\rightarrow$ candidate next node)
- This memory lets us control whether the walk:
 - Backtracks
 - Stays local
 - Moves outward
- Result: a single mechanism that can smoothly shift between BFS-like and DFS-like behavior
- This controlled walk process is the key idea leading to the node2vec algorithm

INTERPRETING P AND Q (THE “WALK STYLE” KNOBS)

- Return parameter p controls backtracking:
 - **low p** → **more** backtracking → more local exploration.
 - **high p** → **discourage** revisits → more exploratory walks.
- In-out parameter q controls outward exploration:
 - $q > 1$ biases toward nodes **near t** → BFS-like (local view).
 - $q < 1$ biases toward nodes **far from t** → DFS-like (outward exploration).
- We tune (p,q) to match role similarity vs community similarity.

THE NODE2VEC ALGORITHM

Algorithm 1 The node2vec algorithm

LearnFeatures (Graph $G = (V; E; W)$, Dimensions d , Walks per node r , Walk length l , Context size k , Return p , In-out q)

$\pi = \text{PreprocessModifiedWeights}(G; p; q)$

$G' = (V; E; \pi)$

Initialize walks to Empty

for iter = 1 to r do

 for all nodes $u \in V$ do

 walk = node2vecWalk(G' ; u ; l)

 Append walk to walks

$f = \text{StochasticGradientDescent}(k, d, \text{walks})$

return f

node2vecWalk (Graph $G' = (V; E; \pi)$, Start node u , Length l)

 Initialize walk to $[u]$

 for walk_iter = 1 to l do

 curr = walk[-1]

$V_{\text{curr}} = \text{GetNeighbors}(\text{curr}, G)$

$s = \text{AliasSample}(V_{\text{curr}}; \pi)$

 Append s to walk

return walk

THE NODE2VEC ALGORITHM

Algorithm 1 The node2vec algorithm

LearnFeatures (Graph $G = (V; E; W)$, Dimensions d ,
Walks per node r , Walk length l , Context size k ,
Return p , In-out q)

```
 $\pi$  = PreprocessModifiedWeights( $G$ ;  $p$ ;  $q$ )  
 $G' = (V; E; \pi)$   
Initialize walks to Empty  
for iter = 1 to  $r$  do  
    for all nodes  $u \in V$  do  
        walk = node2vecWalk( $G'$ ;  $u$ ;  $l$ )  
        Append walk to walks  
 $f$  = StochasticGradientDescent( $k$ ,  $d$ , walks)  
return  $f$ 
```

node2vecWalk (Graph $G' = (V; E; \pi)$, Start node u ,
Length l)

```
Initialize walk to [ $u$ ]  
for walk_iter = 1 to  $l$  do  
    curr = walk[-1]  
    Vcurr = GetNeighbors(curr,  $G'$ )  
     $s$  = AliasSample(Vcurr;  $\pi$ )  
    Append  $s$  to walk  
return walk
```

Inputs: graph $G=(V,E,W)$; embedding size d ; walks per node r walk length l ;
context window k ; walk-bias knobs p,q

Precompute biased transition probabilities

Compute π so each step in a walk can be sampled quickly
 p,q control whether walks stay local, backtrack, or explore outward

Generate a corpus of node sequences (“walks”)

Repeat r times: for every node $u \in V$, start a walk of length l
Each walk is a sequence of nodes, like a “sentence” in NLP
Collect all walks into one big list

How a single walk is created (node2vecWalk)

Start with [u]
For each step: look up neighbors of current node, then sample the next
node using π
Alias sampling is used so each step is fast

Learn embeddings from the walk corpus (Skip-gram training)

Treat walks as sentences; nodes as words
Use window k to define context pairs
Run SGD to learn $f: V \rightarrow \mathbb{R}^d$

Output: f the embedding table (one d -dimensional vector per node)

SKIP-GRAM

- **Skip-gram** : learn vectors so a center item can **predict its context items**
 - In text: center word predicts nearby words in a window
 - In graphs: center node predicts nearby nodes in a *sampled neighborhood* (from walks)
- **Training signal** :
 - If two nodes appear in similar contexts, their vectors should become similar
 - Co-occurrence in walk windows is treated as “relatedness”
- **Objective**:
 - Maximize the probability of observing neighborhood nodes given the source node’s embedding
 - Use dot products between embeddings to score how compatible two nodes are
- **Why it fits graphs**:
 - Graphs don’t give contexts directly, but random walks *create* them
 - Skip-gram then provides a scalable way to turn those contexts into embeddings
- **Essentially**, full normalization over all nodes is expensive → use negative sampling to train efficiently

OBJECTIVE FUNCTION

- Maximum-Likelihood Objective

$$\max_f \sum_{u \in V} \log \Pr(N_S(u) | f(u)).$$

- Probability node n_i is in the context of node u (using softmax)

$$\Pr(n_i | f(u)) = \frac{\exp(f(n_i) \cdot f(u))}{\sum_{v \in V} \exp(f(v) \cdot f(u))}.$$

NODE2VEC ALGORITHM: THREE PHASES

■ Phases

- 1) Preprocess transition probabilities for 2nd-order walks (often via alias sampling for $O(1)$ draws).
- 2) Simulate r random walks of length ℓ starting from each node (walk corpus).
- 3) Train Skip-gram on the walk corpus with window size k , embedding dim d , negative sampling.
- All phases parallelize well; random walks reuse samples efficiently across many nodes.

ALGORITHM

- **Inputs:** graph $G=(V,E,W)$; embedding dimension d ; walks per node r ; walk length l ; context window k ; walk controls p (return) and q (in-out)
- **Step 1: Precompute “biased” transition probabilities**
 - Modify edge transition weights using p,q so walks can be more local or more exploratory
 - Store these as π (so sampling is fast later)
- **Step 2: Generate a corpus of node sequences (“walks”)**
 - Repeat r times: for every node u , simulate one walk of length l starting at u
 - Each walk is a sequence like $[u,v_1,v_2,\dots]$
 - Collect all sequences into a big list called walks
- **Step 3: Learn embeddings from the walk corpus (Skip-gram training)**
 - Treat each walk like a sentence
 - Use window size k to create (center node, context node) training pairs
 - Run SGD to learn $f: V \rightarrow \mathbb{R}^d$ so nodes predict their contexts well
- **Inside node2vecWalk: how one walk is built**
 - Start at u
 - For each step: look at current node’s neighbors and sample the next node using $\pi \setminus p_i \pi$
 - Alias sampling makes each step efficient (constant-time sampling after preprocessing)
- **Output:** the embedding table f (one d -dimensional vector per node)

HYPERPARAMETERS WE ACTUALLY TUNE

- Embedding dim d : capacity vs overfitting; 64–256 typical.
- Walk length ℓ and walks/node r : stability vs compute; more walks lowers variance.
- Context window k : how local the “context” is (like word2vec).
- Negatives per positive: quality vs time.
- p, q : role vs community; grid-search a small set of values.
- Learning rate / epochs: mostly inherited from word2vec training dynamics.

FROM NODE EMBEDDINGS TO EDGE EMBEDDINGS (LINK PREDICTION)

- Many tasks need pair features $g(u,v)$, not just $f(u)$ and $f(v)$.
- node2vec proposes simple binary operators to compose edge vectors:
 - • Average: $(f(u)+f(v))/2$
 - • Hadamard: $f(u) \odot f(v)$ (often strong for link prediction)
 - • Weighted L1: $|f(u)-f(v)|$
 - • Weighted L2: $|f(u)-f(v)|^2$
- Then train a standard classifier (logistic regression, etc.) on these edge features.

WHAT NODE2VEC BUYS YOU (AND WHAT IT DOESN'T)

- Strengths: scalable unsupervised embeddings; tunable to capture homophily vs structural roles; strong baseline for many graph ML tasks.
- Best fit: homogeneous graphs where edges mean “related” without many relation types.
- Limits: relation semantics are not explicit; heterogeneous knowledge graphs often need relation-aware models (e.g., TransE family).
- Takeaway: node2vec is “graph word2vec” with a crucial walk-bias knob (p, q).



MULTI-RELATIONAL DATA



TRANSE:

WHY EMBEDDINGS FOR MULTI-RELATIONAL DATA?

- Knowledge graphs store facts as **triples** (h, r, t)
 - with **many relation types** r.
- Goal: learn vector embeddings for entities and relations so we can score plausible vs implausible triples.
- Primary task: link prediction (complete missing facts), plus clustering and downstream QA.
- Design target: simple, few parameters, scalable to very large KGs.

CORE ASSUMPTION: RELATIONS AS TRANSLATIONS

- For a true triple (h, r, t) , embeddings should satisfy: $h + r \approx t$.
- Interpretation: relation vector r encodes how to move from head to tail.
- Score/energy for a triple: $d(h + r, t)$, using L1 or L2 distance.
- Lower distance $\rightarrow$ more plausible triple; higher distance $\rightarrow$ implausible.

SCORING FUNCTION AND GEOMETRY

- Entity embeddings $h, t \in \mathbb{R}^k$; relation embeddings $r \in \mathbb{R}^k$
- Energy: $d(h + r, t) = \|h + r - t\|_1$ or $\|h + r - t\|_2$
- L1 and L2 Norms

$$\|h + r - t\|_1 = \sum_i |h_i + r_i - t_i|$$

$$\|h + r - t\|_2 = \sqrt{\sum_i (h_i + r_i - t_i)^2}$$

- Geometric intuition: each relation forms a displacement pattern across entity pairs.
- Nearest-neighbor view: given (h, r) , rank candidate tails by proximity to $h+r$.

TRAINING OBJECTIVE: MARGIN RANKING LOSS

- Train to make true triples score better than corrupted triples by a margin γ .
- Negatives created by corrupting head or tail: (h', r, t) or (h, r, t') .
- Loss per pair: $[\gamma + d(h+r, t) - d(h'+r, t')]_+$
- Negative sampling avoids enumerating all corruptions; usually sample one (or a few) negatives per positive.

OPTIMIZATION DETAILS AND CONSTRAINTS

- **Entity vectors are kept the same length ($\|e\|_2 = 1$)**
 - All entity embeddings lie on a unit sphere
 - Prevents the model from “cheating” by just making vectors bigger
 - Forces learning to focus on *relative positions*, not magnitude
- **Relation vectors are free**
 - A relation is a translation (an arrow from head toward tail)
 - Different relations can have different step sizes
 - So we do not force relation vectors to have unit length
- **How training works**
 - Take small batches of true triples
 - For each true triple, create corrupted (false) versions
 - Update vectors so true triples have smaller distance than false ones
- **Main knobs**
 - k : embedding dimension (size of vectors)
 - γ : how strongly true triples must beat false ones
 - Learning rate: how big each update step is
 - L1 vs L2: how distance is measured
 - Negatives per positive: how many false triples we generate per true one

EVALUATION PROTOCOL: RAW VS FILTERED RANKING

- Standard evaluation: replace head (or tail) with all entities, rank candidates by score.
- Metrics: Mean Rank, MRR, Hits@k.
- Filtered setting removes candidates that are actually true triples elsewhere in the KG (so you're not penalized for ranking another true fact higher).
- Filtered metrics are the modern standard for fair link prediction evaluation.

WHY TRANSE WAS A BREAKTHROUGH

- Parameter efficiency: one vector per entity and per relation
 - $O(|E|k + |R|k)$
- Training scales to very large KGs (millions of entities).
- Interpretability: relation vectors are literal translation offsets.
- Strong empirical baseline despite simplicity; launched a large family of translation models.

WHERE TRANSE STRUGGLES (IMPORTANT FAILURE CASES)

- 1-to-many / many-to-1: translations force multiple correct tails (or heads) to cluster near one point.
- Many-to-many relations: can collapse structure; difficult to satisfy all constraints simultaneously.
- Relations requiring true 3-way interactions can be underfit by pure translation scoring.
- Motivated extensions: TransH/TransR/TransD and more expressive bilinear/complex models.

ASSOCIATED CODE

- [Node2Vec illustration notebook](#)
- [TransE \(PyKEEN\) illustration notebook](#)